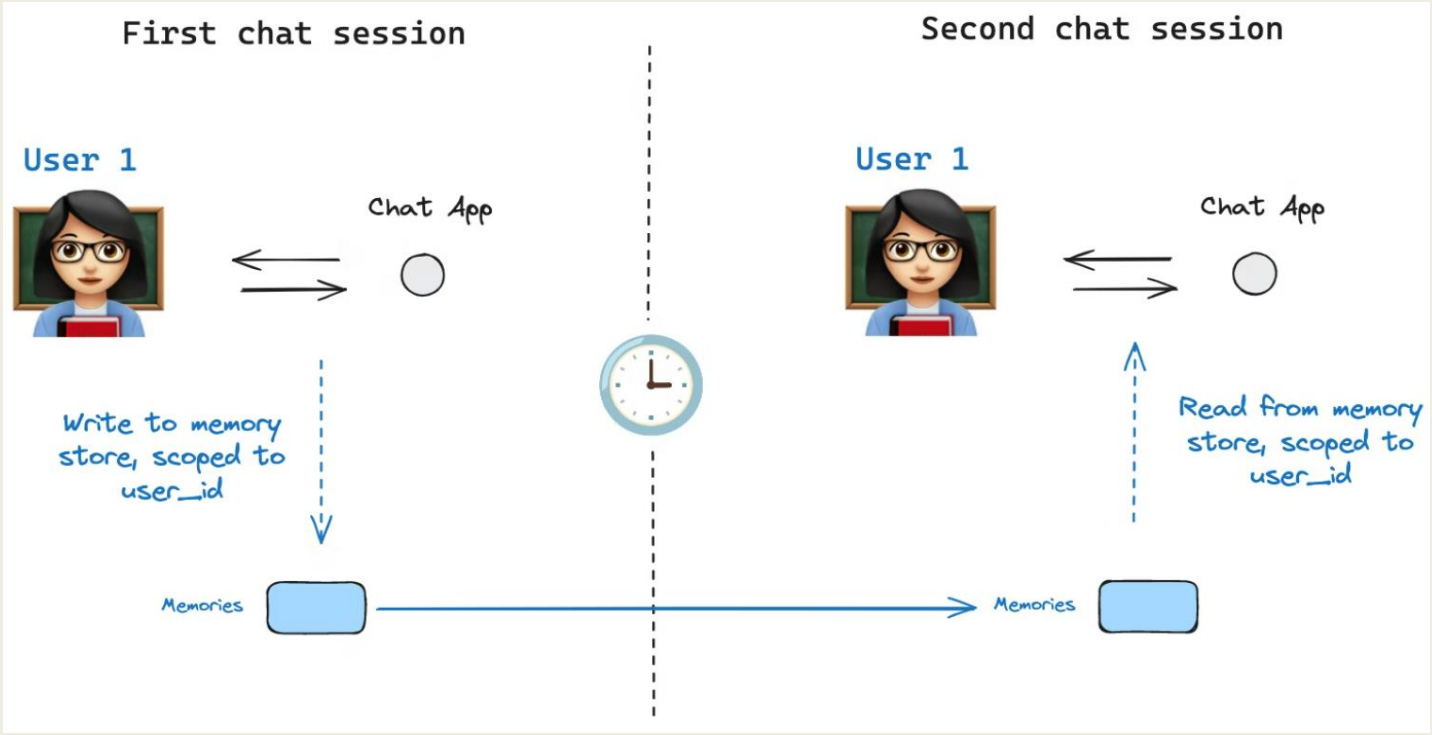
What is Memory Store?

Memory Store in LangGraph is a system that persists long-term memories as JSON documents across multiple conversational threads using custom namespaces (folders) and keys (file names)



Why Memory Store Exists?

Agents need to remember information across multiple conversations, not just within a single thread. It provides a centralized, persistent repository where any agent can store and retrieve shared knowledge using either direct key-based access or semantic similarity search.

WHY ??

- Remembers user preferences, context, and history across multiple chat sessions
- Unlike checkpointers that are thread-specific, stores persist globally
- Multiple agents or threads can access the same memory
- Enables collaborative workflows and team-wide context

Memory Store Fields

- **Namespace** - A hierarchical tuple that defines the scope and organization of stored data, such as `("users", "alice_123", "preferences")`.
- **Key** - A unique identifier within a namespace used to retrieve specific memory items directly.
- **Value -** A dictionary containing the actual data/content to be stored in the memory.

Note: Memory Store also supports additional fields like `query`, `limit`, `filter`, and `score` for advanced search and retrieval operations.

Memory Store Methods

1. get() - Retrieve Memory

- Fetch a specific memory item by namespace and key
- Access stored user data or preferences

```
store = InMemoryStore()

# Get specific user profile
namespace = ("users", "alice_2024", "profile")
result = store.get(namespace, "main_profile")
```

2. search() - Semantic Search

- Find memories based on meaning rather than exact matches
- Retrieve relevant context using natural language queries

```
namespace = ("users", "alice_2024", "conversations")
results = store.search(
    namespace,
    query="machine learning projects",
    limit=5
)
```

Memory Store Methods

3. delete() - Remove Memory

- Delete specific memory entries
- Clean up outdated or sensitive information

```python
# Delete a specific memory
namespace = ("users", "alice_2024", "temp")
store.delete(namespace, "session_data")
```

4. batch() - Bulk Operations

- Find memories based on meaning rather than exact matches
- Retrieve relevant context using natural language queries

```python
# Batch insert/update/delete
operations = [
    Op(namespace=("users", "alice_2024", "settings"),
       key="theme",
       value={"color": "dark", "font": "roboto"}),

    Op(namespace=("users", "alice_2024", "settings"),
       key="notifications",
       value={"email": True, "sms": False}),

    Op(namespace=("users", "alice_2024", "temp"),
       key="old_cache",
       value=None)  # Delete operation
]
```

Semantic Search in Memory Store

What is Semantic Search?

Semantic search allows agents to find relevant memories based on meaning, not just exact matches, making it easier to retrieve contextual information across conversations.

Key Difference:

- **Keyword Search**: Matches exact text → "vegetarian food"
- **Semantic Search**: Matches meaning → "What does the user like to eat?" retrieves "User prefers vegetarian food"

How to Configure Memory Store:

1.Initialize the store with an embedding model for semantic search:

```python
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": "openai:text-embedding-3-small",  # embedding model
        "dims": 1536,                              # embedding dimensions
    }
)
```

2. Store memories name spaced per user:

```python
# Save a memory
store.put(
    ("user_123", "memories"),        # namespace — scoped to user + category
    key="food_preference",           # unique key for this memory
    value={"text": "User prefers vegetarian food"}
)
```

Semantic Search in Memory Store

3.Wire the store into your LangGraph agent:

```python
from langgraph.checkpoint.memory import MemorySaver

graph = builder.compile(
    checkpointer=MemorySaver(),    # persists conversation state (short-term)
    store=store                     # persists memories across conversations (long-term)
)
```

checkpointer = short-term — remembers within a single conversation thread (chat history, intermediate state)

store = long-term — remembers across conversations (user preferences, facts, past interactions)

For search :

```python
# Search using natural language query
memories = store.search(
    ("user_123", "memories"),
    query="I like food?",
    limit=5
)
```